React Hooks

Programação Web 2

Objetivos de Aprendizagem

- Compreender o funcionamento de Hooks
- Aplicar Hooks nas interfaces web

Agenda

- Hooks
- `useState`
- `useEffect`
- `useContext`
- `useRef`
- `useReducer`
- `useCallback`
- `useMemo`

React Hooks

React Hooks

> Hooks permitem que funções acessem o estado e outras funções do React sem utilizar classes

React Hooks

Tipos

- State hooks
- Context hooks
- Ref hooks
- Effect hooks
- Performance hooks
- Custom

State Hooks

> O estado permite que um componente "lembre" informações como a entrada do usuário. Por exemplo, um componente de formulário pode usar o estado para armazenar o valor de entrada, enquanto um componente de galeria de imagens pode usar o estado para armazenar o índice da imagem selecionada.

- `useState` declara uma variável de estado que você pode atualizar diretamente
- `useReducer` declara uma variável de estado com a lógica de atualização dentro de uma função `reducer`

Context Hooks

`useContext`

> Permite que um componente receba informações de componentes pais distantes sem precisar passá-las como propriedades. Por exemplo, o componente de nível superior do seu aplicativo pode passar o tema da interface do usuário atual para todos os componentes abaixo dele, independentemente da profundidade na hierarquia.

```
1  function Button() {
2    const theme = useContext(ThemeContext);
3    // ...
```

Ref Hooks

> As refs permitem que um componente armazene informações que não são usadas para renderização, como um nó DOM ou um ID de tempo limite. Ao contrário do estado, atualizar uma `ref` não renderiza o componente novamente. As refs são uma "saída de emergência" do paradigma React. Elas são úteis quando você precisa trabalhar com sistemas que não são do React, como as APIs nativas do navegador.

- `useRef` declara uma referência. Você pode armazenar qualquer valor nela, mas geralmente é usada para armazenar um nó DOM.
- `useImperativeHandle` permite personalizar a referência exposta pelo seu componente. Isso raramente é usado.

Effect Hooks

> Os efeitos permitem que um componente se conecte e sincronize com sistemas externos. Isso inclui lidar com rede, DOM do navegador, animações, widgets escritos usando uma biblioteca de interface do usuário diferente e outros códigos que não são do React.

- `useEffect` conecta um componente a um sistema externo

Performance Hooks

> Uma maneira comum de otimizar o desempenho de renderização é evitar tarefas desnecessárias. Por exemplo, você pode instruir o React a reutilizar um cálculo em cache ou a ignorar uma nova renderização se os dados não tiverem sido alterados desde a renderização anterior.

- `useMemo` permite armazenar em cache o resultado de um cálculo custoso
- `useCallback` permite armazenar em cache a definição de uma função antes de passá-la para um componente otimizado
- `useTransition` permite marcar uma transição de estado como não bloqueante e permitir que outras atualizações a interrompam

useState

`useState`

- Chamada nas declarações iniciais de um componente para declarar uma variável de **estado**
- A convenção é nomear variáveis de estado como `[something, setSomething]` usando desestruturação de *array*

```
1  import { useState } from 'react';
2
3  function MyComponent() {
4    const [age, setAge] = useState(28);
5    const [name, setName] = useState('Taylor');
6    const [todos, setTodos] = useState(() => createTodos());
7    // ...
```

`useState`

- `useState` retorna um array com dois valores:
  - O estado atual. Durante a primeira renderização, ele corresponderá ao `initialState` passado
  - A função `set` que permite atualizar o estado para um valor diferente e acionar uma nova renderização
- `useState` não pode ser chamado em *loops* ou condicionais

Funções `set`

- Funções `set` retornadas por `useState` permitem atualizar o estado para um valor diferente e acionar uma nova renderização
- Não retornam valor
- É possível passar o próximo estado diretamente ou uma função que o calcula a partir do estado anterior:

```
1  const [name, setName] = useState('Edward');
2
3  function handleClick() {
4    setName('Taylor');
5    setAge(a => a + 1);
6    // ...
```

`useState`

Utilização típica

Basic useState examples

useEffect

`useEffect`

> Permite realizar ações complementares, tais como buscar dados, atualizar o DOM, disparar *timers*, etc nos componentes.

`useEffect(function, dependency)`

- Roda a cada renderização
- Utiliza dois argumentos
  - `function` define o que será executado
  - `dependency` variável de controle

useEffect

Exemplo sem dependência

```
1   import { useState, useEffect } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   function Timer() {
5     const [count, setCount] = useState(0);
6
7     useEffect(() => {
8       setTimeout(() => {
9         setCount((count) => count + 1);
10      }, 1000);
11    });
12
13    return <h1>I've rendered {count} times!</h1>;
14  }
15
16  createRoot(document.getElementById('root')).render(
17    <Timer />
18  );
```

Rodar Exemplo

`useEffect`

Exemplo sem dependência

1. `useEffect` roda a cada renderização
2. A cada modificação do contador uma renderização acontece, chamando novamente `useEffect`
3. O contador é modificado novamente e sem limites

> Como controlar quando `useEffect` roda?

`useEffect`

Renderização inicial

```
1   import { useState, useEffect } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   function Timer() {
5     const [count, setCount] = useState(0);
6
7     useEffect(() => {
8       setTimeout(() => {
9         setCount((count) => count + 1);
10      }, 1000);
11    }, []);
12
13    return <h1>I've rendered {count} times!</h1>;
14  }
15
16  createRoot(document.getElementById('root')).render(
17    <Timer />
18  );
```

Rodar Exemplo

useEffect

Dependência de uma variável

```
1   import { useState, useEffect } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   function Counter() {
5     const [count, setCount] = useState(0);
6     const [calculation, setCalculation] = useState(0);
7
8     useEffect(() => {
9       setCalculation(() => count * 2);
10    }, [count]);
11
12    return (
13      <>
14        <p>Count: {count}</p>
15        <button onClick={() => setCount((c) => c + 1)}>+</button>
16        <p>Calculation: {calculation}</p>
17      </>
18    );
19  }
20
21  createRoot(document.getElementById('root')).render(
```

useContext

`useContext`

> O React Context é uma forma de gerenciar o estado **globalmente**. Pode ser usado em conjunto com `useState` para compartilhar o estado entre componentes aninhados

`useContext`

Prop drilling

- O estado deve ser mantido pelo componente pai de nível mais alto na pilha que necessita acessar o estado
- Imagine que temos vários componentes aninhados, logo os componentes no topo e na base da pilha precisam acessar o estado
- Para fazer isso sem usar o Contexto, é necessário passar o estado como `props` através de cada componente aninhado (*prop drilling*)

useContext

Exemplo de *Prop drilling*

```
1   import { useState } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   function Component1() {
5     const [user, setUser] = useState("Linus");
6
7     return (
8       <>
9         <h1>{`Hello ${user}!`}</h1>
10        <Component2 user={user} />
11      </>
12    );
13  }
14
15  function Component2({ user }) {
16    return (
17      <>
18        <h1>Component 2</h1>
19        <Component3 user={user} />
20      </>
21    );
22  }
```

useContext

Exemplo com hook

```
1   import { useState, createContext, useContext } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   const UserContext = createContext();
5
6   function Component1() {
7     const [user, setUser] = useState("Linus");
8
9     return (
10      <UserContext.Provider value={user}>
11        <h1>{`Hello ${user}!`}</h1>
12        <Component2 />
13      </UserContext.Provider>
14    );
15  }
16
17  function Component2() {
18    return (
19      <>
20        <h1>Component 2</h1>
21        <Component3 />
22      </>
```

useRef

`useRef`

> `useRef` permite persistir valores entre renderizações

`useRef`

- Pode ser usado para armazenar um valor mutável que não causa uma nova renderização quando atualizado
- Para acessar um elemento DOM diretamente
- Retorna o objeto `current`

```
import { useState, useRef, useEffect } from 'react';
import { createRoot } from 'react-dom/client';

function App() {
  const [inputValue, setInputValue] = useState("");
  const count = useRef(0);

  useEffect(() => {
    count.current = count.current + 1;
  });

  return (
    <>
      <p>Type in the input field:</p>
      <input
        type="text"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
      <h1>Render Count: {count.current}</h1>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <App />
);
```

```
1   import { useRef } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   function App() {
5     const inputElement = useRef();
6
7     const focusInput = () => {
8       inputElement.current.focus();
9     };
10
11    return (
12      <>
13        <input type="text" ref={inputElement} />
14        <button onClick={focusInput}>Focus Input</button>
15      </>
16    );
17  }
18
19  createRoot(document.getElementById('root')).render(
20    <App />
21  );
```

Rodar Exemplo

useReducer

`useReducer`

> Semelhante a `useState`, porém pode aplicar uma lógica mais apurada

`useReducer(reducer, initialState, init)`

- `reducer` função com a lógica personalizada
- `initialState` objeto de inialização
- `init` incializa o estado (opcional)

useReducer

Exemplo

```
1   import { useReducer } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   const initialScore = [
5     {
6       id: 1,
7       score: 0,
8       name: "John",
9     },
10    {
11      id: 2,
12      score: 0,
13      name: "Sally",
14    },
15  ];
16
17  const reducer = (state, action) => {
18    switch (action.type) {
19      case "INCREASE":
20        return state.map((player) => {
21          if (player.id === action.id) {
22            return { ...player, score: player.score + 1 };
```

`useCallback` e `useMemo`

`useMemo`

> Usado para memorizar (armazenar em *cache*) um valor e retornar

`useCallback`

> Usado para memorizar uma função de retorno de chamada

`useCallback(callback, dependencies)`

- `callback` função a ser armazenada
- `dependencies` um array contendo variáveis de dependências. Caso uma das dependências mude, a função `callback` é executada novamente

Sem `useCallback`

```
1   import React, { useState } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   // Child component that receives a function prop
5   const Button = React.memo(({ onClick, text }) => {
6     alert(`Child ${text} button rendered`);
7     return <button onClick={onClick}>{text}</button>;
8   });
9
10  // Parent component without useCallback
11  function WithoutCallbackExample() {
12    const [count1, setCount1] = useState(0);
13    const [count2, setCount2] = useState(0);
14
15    // This function is recreated on every render
16    const handleClick1 = () => {
17      setCount1(count1 + 1);
18    };
19
20    const handleClick2 = () => {
21      setCount2(count2 + 1);
22    };
```

Aplicando `useCallback`

```
1   import React, { useState, useCallback } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   // Child component that receives a function prop
5   const Button = React.memo(({ onClick, text }) => {
6     console.log(`${text} button rendered`);
7     return <button onClick={onClick}>{text}</button>;
8   });
9
10  // Parent component with useCallback
11  function WithCallbackExample() {
12    const [count1, setCount1] = useState(0);
13    const [count2, setCount2] = useState(0);
14
15    // These functions are memoized and only recreated when dependencies change
16    const handleClick1 = useCallback(() => {
17      setCount1(count1 + 1);
18    }, [count1]);
19
20    const handleClick2 = useCallback(() => {
21      setCount2(count2 + 1);
22    }, [count2]);
```

useMemo

`useMemo`

> Pode ser usado para evitar que funções dispendiosas e que consomem muitos recursos sejam executadas desnecessariamente.

Sem `useMemo`

```
1   import { useState } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4
5   const App = () => {
6     const [count, setCount] = useState(0);
7     const [todos, setTodos] = useState([]);
8     const calculation = expensiveCalculation(count);
9
10    const increment = () => {
11      setCount((c) => c + 1);
12    };
13    const addTodo = () => {
14      setTodos((t) => [...t, "New Todo"]);
15    };
16
17    return (
18      <div>
19        <div>
20          <h2>My Todos</h2>
21          {todos.map((todo, index) => {
22            return <p key={index}>{todo}</p>;
```

Rodar Exemplo

Aplicando `useMemo`

```
1   import { useState, useMemo } from 'react';
2   import { createRoot } from 'react-dom/client';
3
4   const App = () => {
5     const [count, setCount] = useState(0);
6     const [todos, setTodos] = useState([]);
7     const calculation = useMemo(() => expensiveCalculation(count), [count]);
8
9     const increment = () => {
10      setCount((c) => c + 1);
11    };
12    const addTodo = () => {
13      setTodos((t) => [...t, "New Todo"]);
14    };
15
16    return (
17      <div>
18        <div>
19          <h2>My Todos</h2>
20          {todos.map((todo, index) => {
21            return <p key={index}>{todo}</p>;
22          })}
```

Rodar Exemplo

Perguntas?

Exercício

1

Criar um Blog Pessoal utilizando React. Preencha o conteúdo com dados "fake" gerados por APIS como JSONPlaceholder ou similar.

Criar um blog com as funcionalidades:

- Página inicial com lista de artigos
- Página de artigo individual
- Página sobre
- Formulário de contato
- Componentes reutilizáveis
- Dados em JSON ou API (JSONPlaceholder)
- Responsividade

Referências

- Bulit-in React Hooks
- W3Schools React Hooks
- JSONPlaceholder

José Roberto Bezerra

jbroberto@ifce.edu.br

jbroberto76

pw2-repo

Powered by Slidev

Cover image by haikei